Pattern Matching and Switch Expressions

7 min read · Updated 28 Jul 2026 · Topic 50 of 65

Bookmark · Mark done

Why it matters

Pattern matching is the feature that most changes how modern Java *reads*. The instanceof-cast-use dance, which appears in essentially every Java codebase written before 2021, collapses into one line — and combined with sealed types it turns a category of runtime bug into a compile error. It is also the clearest signal in an interview that you have followed the language past Java 8.

The concept, simply

Three related features that arrived together:

- **Pattern matching for** `instanceof` (Java 16) — test and bind in one step.
- **Switch expressions** (Java 14) — `switch` that produces a value, with arrow labels and no fall-through.
- **Pattern matching for** `switch` (Java 21) — switch on *types*, with guards and record deconstruction.

Code examples

JAVA

Copy

```
// ✗ The old dance — the type is written three times.
if (obj instanceof String) {
    String s = (String) obj;
    if (s.length() > 5) return s.toUpperCase();
}

// ✓ Pattern matching: test, cast and bind in one, and "s" is in scope
```

```java
// The scoping is genuinely clever — this works too:
if (!(obj instanceof String s)) return "not a string";
return s.toUpperCase();          // ✓ s IS in scope here — the compiler
                                 //   knows the negated branch returned
```

Switch expressions, which fix three long-standing problems at once:

JAVA Copy

```java
// ✗ Old: fall-through by default, mutation, easy to forget a break.
String label;
switch (day) {
    case SATURDAY:
    case SUNDAY: label = "weekend"; break;
    default: label = "weekday";
}

// ✓ New: an expression, no fall-through, exhaustive over an enum so no
//   default is needed — and adding an enum constant breaks the build.
String label = switch (day) {
    case SATURDAY, SUNDAY -> "weekend";
    case MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY -> "weekday";
};

// yield for a multi-statement branch:
int size = switch (kind) {
    case SMALL -> 1;
    case LARGE -> {
        log.info("large requested");
        yield computeLarge();
    }
};
```

Pattern matching for switch (Java 21) — the payoff:

JAVA Copy

```java
sealed interface Shape permits Circle, Square, Rectangle { }
record Circle(double r) implements Shape { }
```

```java
// Type patterns, no default needed because Shape is sealed:
double area(Shape s) {
    return switch (s) {
        case Circle c    -> Math.PI * c.r() * c.r();
        case Square q    -> q.side() * q.side();
        case Rectangle r -> r.w() * r.h();
    };
}

// Guards with "when", and record DECONSTRUCTION patterns:
String describe(Shape s) {
    return switch (s) {
        case Circle c when c.r() > 100    -> "huge circle";
        case Circle c                     -> "circle of radius " + c.r();
        case Rectangle(double w, double h) when w == h -> "square-ish " +
        case Rectangle(double w, double h) -> w + " by " + h;
        case Square q                     -> "square " + q.side();
    };
}
// The Rectangle case destructures the record straight into w and h — no
// accessor calls at all. Order matters: guarded cases must come first,
// and the compiler rejects a case that is unreachable.

// Null is now handleable explicitly instead of throwing:
switch (obj) {
    case null      -> "nothing";
    case String s  -> "string " + s;
    default        -> "other";
}
```

Where this replaces a real pattern — the visitor:

JAVA Copy

```java
// The visitor pattern existed because Java could not switch on type safel
// A sealed hierarchy plus pattern-matching switch does the same job with
// none of the ceremony, and adding a case still fails at compile time:
```

Under the hood

Flow scoping is what makes `instanceof` patterns pleasant. The bound variable is in scope precisely where the compiler can prove the test succeeded — including after an early return in the negated branch. It is not a simple block scope; it follows the control flow, which is why the "return if not a String, then use `s`" example compiles.

Pattern switches compile to `invokedynamic` against a bootstrap that performs the type tests, rather than to a chain of `instanceof` checks. That leaves room for the JIT to optimise dispatch and for the implementation to improve without recompiling your code — the same strategy used for lambdas and for record `equals`.

Exhaustiveness is a compile-time proof, with a runtime guard. When a switch over a sealed type covers every permitted subtype, no `default` is required. The compiler still emits a synthetic fallback that throws `MatchException`, to cover the case where the hierarchy was recompiled separately after the switch — a genuinely subtle detail worth knowing.

One caution: **adding a `default` branch to a sealed switch silently disables exhaustiveness checking.** The compiler will no longer tell you about a new case, because you claimed to handle everything. Omitting `default` on a sealed type is not a stylistic choice — it is what buys you the safety.

Common mistakes

- **Adding `default` to a sealed switch.** Disables the exhaustiveness check that is the whole point.
- **Ordering guarded cases after unguarded ones.** The unguarded case matches first; the compiler rejects the unreachable one.
- **Mixing `:` and `→` in one switch.** Not allowed — pick one form.
- **Forgetting `yield` in a block branch.** An arrow branch with braces must yield a value.

- **Still writing the visitor pattern.** Sealed types plus pattern switch do it with far less code.

Interview lens

Q: "What is pattern matching for `instanceof`?"
A: It combines the type test, the cast and the variable binding into one expression — `if (obj instanceof String s)` — so the type is written once instead of three times. The binding uses flow scoping: `s` is in scope wherever the compiler can prove the test succeeded, which includes the code after an early return in the negated branch.

Q: "What do switch expressions improve?"
A: Three things. They produce a value, so you stop declaring a variable and mutating it. Arrow labels remove fall-through, which eliminates the forgotten-`break` bug. And a switch expression over an enum must be exhaustive, so adding a constant breaks the build rather than silently falling to `default`. `yield` returns a value from a multi-statement branch.

Q: "What does pattern matching for switch add in Java 21?"
A: Switching on types rather than constants, with `when` guards for conditions and record deconstruction patterns that destructure components straight into variables. Combined with sealed types the switch is exhaustive with no `default`, so adding a new subtype is a compile error in every switch that does not handle it. It also lets you write `case null` instead of the classic switch's NPE.

Q: "Why should you avoid `default` with sealed types?"
A: Because it turns off exhaustiveness checking. Once you claim to handle everything, the compiler stops telling you about new subtypes, and the new case

silently falls into `default` — usually doing the wrong thing at run time. Omitting `default` is what converts "I hope I updated every switch" into a build failure.

Q: "Does this replace the visitor pattern?"

A: For most uses, yes. Visitor existed because Java had no safe way to switch on type and no way to force handling of every case. A sealed hierarchy with a pattern-matching switch gives both, without the accept methods, the visitor interface or the double dispatch. Visitor still has a place when the operations must be extensible by third parties without touching the hierarchy.

Cheat-card

- `obj instanceof String s` — test, cast, bind in one. **Flow scoping**, so it works after an early return.
- Switch **expressions**: arrow labels, **no fall-through**, produce a value, `yield` in blocks.
- Java 21 pattern switch: **type patterns** · `when` **guards** · **record deconstruction** · `case null`.
- Sealed + patterns ⇒ **exhaustive, no** `default`. Adding a case = compile error.
- **Adding** `default` **disables exhaustiveness** — do not.
- Guarded cases must precede unguarded ones; unreachable cases are rejected.
- Compiles to `invokedynamic`; a synthetic `MatchException` guards separate recompilation.
- Largely **retires the visitor pattern**.

QUICK CHECK — no marks, just for you.

1. After 'if (!(obj instanceof String s)) return "no";' can you use s on the next line?

- ○ Yes — flow scoping puts s in scope wherever the compiler can prove the test succeeded, including after the negated branch returns
- ○ No — the binding is only in scope inside the if block

- Only if s is declared final
- No — it is a compile error to negate an instanceof pattern

Check answer

2. Why should you NOT add a default branch to a switch over a sealed type?

- It makes the switch slower
- It disables exhaustiveness checking — the compiler stops warning about new subtypes because you claimed to handle everything, so a new case silently falls into default
- default is not allowed in switch expressions
- It causes a compile error

Check answer

3. What do switch EXPRESSIONS improve over classic switch statements?

- They are evaluated at compile time
- They allow fall-through between cases by default
- They produce a value, arrow labels remove fall-through, and over an enum they must be exhaustive — so adding a constant breaks the build
- They can switch on any object type without patterns

Check answer

Records and Sealed Types

Next
Virtual Threads and Structured Concurrency

LEARN MORE

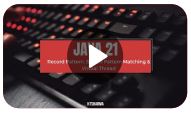
ARTICLES & DOCS

- Pattern matching for instanceof (JEP 394)
  openjdk.org
- Pattern matching for switch (JEP 441)
  openjdk.org
- Record patterns (JEP 440)
  openjdk.org

VIDEOS



- Modern Java Course - Pattern Matching in Switch & Sealed Classes
  Debug Agent



- Java 21 : Record Pattern, Switch Pattern Matching and Virtual Thread
  Himanshu Sharma

Hand-picked references for this lesson.